

Master Programmer Curriculum

A 6-Month Journey into Flutter, Full-Stack Web & AI

Designed for 5th Grade

Target Age: 10-11 Years Old
Pace: 2 Classes / Week (1 Hour Each)

Curriculum Overview

This 24-week curriculum is specifically designed to transition a bright 10-11 year old student from basic coding concepts (C++, HTML) into professional-grade development. By utilizing an **AI-first, project-based approach**, heavy theoretical concepts are translated into fun, visual, and interactive challenges.

- **Phase 1 (Months 1-3):** Solidifying Flutter app development, introducing AI APIs, and translating core CS Fundamentals (OOP, SOLID) into kid-friendly analogies. Transitioning to web concepts via Next.js.
- **Phase 2 (Months 4-6):** Mastering Full-Stack Web Development (Node.js, MongoDB) and introducing foundational Machine Learning using Python and Teachable Machine, concluding with a massive Capstone Project.

PHASE 1: APP MASTERY & CS FOUNDATIONS (MONTHS 1-3)

Week 1: Flutter & AI Superpowers - The Basics

Session 1 | Welcome Back! Flutter + AI APIs

Goal: Understand what an API is and how to talk to an AI from a Flutter app.

- Review: Flutter Widgets (Row, Column, Container).
- Concept: What is an API? (The 'Restaurant Waiter' analogy).
- Code: Making a basic HTTP request in Dart to fetch data.

Action Item: Mini-Task: Build an app screen with a 'Summon AI' button that prints an AI response to the console.

Session 2 | Displaying AI Data & UI Polish

Goal: Learn to parse JSON data and display it beautifully in the app.

- Concept: What is JSON? (Dictionaries/Maps for computers).
- Code: Parsing JSON in Dart.
- UI: Creating custom Cards to display the AI's response.

Action Item: Homework: Finish the 'Joke Generator' app where pressing a button fetches and displays a random joke from an AI.

Week 2: Building an AI Chatbot App

Session 3 | State Management Basics

Goal: Understand how to update the app's screen when data changes.

- Concept: App State (Memory).
- Code: Using StatefulWidget and setState().
- UI: Building a scrolling chat interface (ListView.builder).

Action Item: Mini-Task: Create a mock chat screen where typing a message adds it to a list.

Session 4 | Connecting the Chatbot

Goal: Integrate the AI API into the chat interface for a real conversation.

- Code: Sending user text to the AI API.
- Code: Handling loading states (showing a spinner).
- Debugging common API errors.

Action Item: Homework: Complete 'My AI Buddy' app and customize its visual theme and avatar.

Week 3: Hardware & Sensors in Flutter

Session 5 | Using the Camera

Goal: Learn to access device hardware securely.

- Concept: App Permissions (Why apps ask for camera access).
- Code: Using the Flutter Camera or Image Picker package.
- UI: Displaying the captured image.

Action Item: Mini-Task: Build a 'Selfie Taker' screen that shows the picture you just took.

Session 6 | AI Image Recognition

Goal: Send images to an AI model to analyze what's in the picture.

- Code: Converting images to Base64 (compressing for the internet).
- Code: Sending images via API to Gemini Vision (or similar).
- UI: Displaying the AI's analysis.

Action Item: Homework: Build the 'What is this?' app where you snap a photo and the AI describes it.

Week 4: Phase 1 Assessment & App Polish

Session 7 | Polishing the App

Goal: Learn how to make an app feel professional.

- UI: Adding animations (Hero widgets, simple transitions).
- Code: Adding an App Icon and customizing the splash screen.
- Refactoring: Organizing folders cleanly.

Action Item: Mini-Task: Add a cool app icon and custom colors to the 'What is this?' app.

Session 8 | Boss Fight: Project Presentation

Goal: Assess understanding, review code, and build presentation skills.

- Review of Flutter concepts learned so far.
- Student presents their finished AI app.
- Code review: Finding areas to improve or optimize.

Action Item: Challenge: Add one custom 'secret' feature to the app before the presentation.

Week 5: CS Fundamentals: Building with Blueprints (OOP)

Session 9 | Classes and Objects

Goal: Understand Object-Oriented Programming basics.

- Concept: Classes as blueprints/cookie-cutters; Objects as the actual cookies.
- Code: Writing simple classes with properties and methods in Python/Dart.
- Analogy: The 'Car' factory.

Action Item: Mini-Task: Code a 'Pokemon' class and create 3 different Pokemon objects.

Session 10 | Inheritance (Family Trees)

Goal: Learn how classes can share and inherit traits.

- Concept: Parent and Child classes.
- Code: Extending classes (e.g., Animal -> Dog, Cat).
- Overriding methods (Dogs bark, Cats meow).

Action Item: Homework: Build an 'Animal Kingdom' program with different animals inheriting from a base class.

Week 6: CS Fundamentals: Advanced OOP

Session 11 | Encapsulation (Secret Agents)

Goal: Learn to protect data inside classes.

- Concept: Public vs. Private variables.
- Code: Getters and Setters.
- Analogy: A bank account balance shouldn't be directly editable.

Action Item: Mini-Task: Create a 'Bank Account' class where money can only be changed via deposit() and withdraw() methods.

Session 12 | Polymorphism (Shape Shifting)

Goal: Understand how different objects can be treated the same way.

- Concept: Calling the same method on different objects.
- Code: Lists of different child objects executing their unique versions of a method.
- Analogy: Pressing 'Play' on different media players.

Action Item: Homework: Create a 'Game Character' system where 'Warrior', 'Mage', and 'Archer' all have a unique attack() method.

Week 7: The Super Rules of Coding (SOLID)

Session 13 | Single Responsibility & Open/Closed

Goal: Learn to keep code clean and modular.

- S: Single Responsibility (A chef cooks, the waiter serves. Don't mix them!).
- O: Open/Closed (Adding plugins to a game without breaking the core game).
- Refactoring messy code examples.

Action Item: Mini-Task: Take a messy 'God Class' and split it into three smaller, focused classes.

Session 14 | Liskov, Interface & Dependency

Goal: Complete the SOLID principles with kid-friendly analogies.

- L: Liskov (If it looks like a duck but needs batteries, it's not a real duck).
- I: Interface Segregation (Don't give a TV remote 100 useless buttons).
- D: Dependency Inversion (Using batteries instead of hardwiring to the wall).

Action Item: Homework: Quiz & Code Challenge: Identify which SOLID rule is being broken in 3 code snippets.

Week 8: Design Patterns for Kids

Session 15 | The Factory Pattern

Goal: Learn how to generate objects smartly.

- Concept: A toy factory that makes toys based on what you ask for.
- Code: Creating a Factory function that returns different subclasses.
- When to use this pattern.

Action Item: Mini-Task: Code a 'VehicleFactory' that returns a Car, Bike, or Plane based on user input.

Session 16 | The Singleton Pattern

Goal: Understand how to ensure only ONE instance of an object exists.

- Concept: The 'President' or 'Game Scoreboard' (There can be only one).
- Code: Implementing a Singleton class.
- Review: Why global variables can be bad.

Action Item: Homework: Build a 'GameManager' Singleton that keeps track of the player's high score across the program.

Week 9: Intro to Web: Next.js & React

Session 17 | Welcome to React & Next.js

Goal: Transition from App Dev to Web Dev.

- Concept: What is a web framework? Component-based architecture.
- Code: Creating a new Next.js project.
- Code: Writing simple functional components.

Action Item: Mini-Task: Build a simple 'Hello World' profile page in Next.js.

Session 18 | Props and State in React

Goal: Make web pages interactive.

- Concept: Props (passing data to components like giving a worker instructions).
- Code: useState() hook (React's memory).
- UI: Buttons that change the text on the screen.

Action Item: Homework: Build an interactive 'Counter & Color Changer' component on your webpage.

Week 10: Styling and Routing in Next.js

Session 19 | Styling with Tailwind CSS

Goal: Make the web page look awesome quickly.

- Concept: Utility-first CSS.
- Code: Adding basic Tailwind classes (flex, p-4, text-blue-500).
- UI: Building a pretty 'Card' component.

Action Item: Mini-Task: Style a set of 3 'Hobby' cards using Tailwind CSS.

Session 20 | Multi-Page Websites (Routing)

Goal: Learn to navigate between different pages.

- Concept: File-based routing in Next.js.
- Code: Creating /about and /contact pages.
- Code: Using the component.

Action Item: Homework: Build a 'Personal Portfolio' site with a Home, About Me, and Projects page.

Week 11: The Backend: Node.js Magic

Session 21 | What is a Server?

Goal: Understand the 'brain' behind the website.

- Concept: Frontend vs. Backend (The store window vs. the stockroom).
- Code: Setting up a basic Node.js/Express server.
- Code: Making a simple 'Hello from Server' route.

Action Item: Mini-Task: Start a local server that responds with 'Welcome to my backend!'.

Session 22 | Building an API

Goal: Create your own API that the frontend can call.

- Concept: RESTful APIs (GET vs. POST).
- Code: Creating a route that returns JSON data (e.g., a list of favorite movies).
- Testing the API using browser or Postman/Thunder Client.

Action Item: Homework: Build a 'Joke API' server that returns a random joke from an array when accessed.

Week 12: Databases: Intro to MongoDB

Session 23 | Databases as File Cabinets

Goal: Understand how data is saved permanently.

- Concept: SQL vs NoSQL (Spreadsheets vs. Documents).
- Tool: Setting up MongoDB Atlas (Cloud).
- Concept: Collections and Documents.

Action Item: Mini-Task: Create a free MongoDB Atlas account and set up a cluster.

Session 24 | Connecting Node to MongoDB

Goal: Make the server talk to the database.

- Code: Using Mongoose to connect Node to Mongo.
- Code: Creating a 'Schema' (rules for the data).
- Code: Saving a simple piece of data to the cloud.

Action Item: Homework: Connect your Joke API to MongoDB and save one joke to the database.

PHASE 2: FULL-STACK WEB & APPLIED AI (MONTHS 4-6)

Week 13: Full Stack Mastery: CRUD

Session 25 | Create & Read (CR)

Goal: Learn to save and display data from the database.

- Code: POST requests (Create).
- Code: GET requests (Read).
- Connecting the Next.js frontend to the Node.js API to display data.

Action Item: Mini-Task: Build a 'Quest Board' app where you can add a new quest and see all quests.

Session 26 | Update & Delete (UD)

Goal: Learn to modify and remove data.

- Code: PUT/PATCH requests (Update).
- Code: DELETE requests (Delete).
- UI: Adding 'Edit' and 'Delete' buttons to the Quest Board.

Action Item: Homework: Complete the CRUD Quest Board. Ensure quests can be marked as 'Done' and deleted.

Week 14: Web Security & Authentication Basics

Session 27 | Logins & Passwords

Goal: Understand how websites keep users safe.

- Concept: Authentication vs. Authorization.
- Concept: Why we NEVER save plain text passwords (Hashing).
- Code: Simulating a simple login check.

Action Item: Mini-Task: Create a 'Secret Page' in Next.js that requires a specific passcode to enter.

Session 28 | Cookies & Tokens

Goal: Learn how websites remember you.

- Concept: What is a Cookie? What is a JWT (JSON Web Token)?
- Analogy: A theme park wristband.
- Implementing simple session memory.

Action Item: Homework: Add a 'Logout' button that clears the session and kicks the user back to the homepage.

Week 15: Web Capstone Project Setup

Session 29 | Planning the App

Goal: Learn software architecture and planning.

- Brainstorming: What will the app do? (e.g., A digital journal, a high-score tracker).
- Design: Drawing wireframes.
- Database design: What schemas are needed?

Action Item: Mini-Task: Draw the UI and write down the Database Schema on paper or Figma.

Session 30 | Building the Foundation

Goal: Set up the full stack environment completely.

- Code: Init Next.js, init Node server, connect MongoDB.
- Code: Build the foundational API routes.
- Code: Scaffold the frontend pages.

Action Item: Homework: Complete the basic routing and API connection for the capstone web project.

Week 16: Web Hosting & Mid-Term Demo

Session 31 | Deploying to the Internet

Goal: Publish the website so the whole world can see it.

- Tool: Introduction to Vercel and Render.
- Code: Connecting GitHub repos for automatic deployment.
- Handling Environment Variables (API keys in the cloud).

Action Item: Mini-Task: Deploy the frontend to Vercel and the backend to Render.

Session 32 | Mid-Term Boss Fight

Goal: Present the deployed full-stack application.

- Code Review: Walking through the full stack.
- Live Demo: Using the live URL.
- Feedback: Identifying future improvements.

Action Item: Challenge: Present your live site to the tutor and explain how the data travels from the button click to the database.

Week 17: Intro to AI & Teachable Machine

Session 33 | How AI Actually Works

Goal: Demystify AI and understand basic concepts.

- Concept: Rules-based programming vs. Machine Learning (Giving a recipe vs. letting it taste the food).
- Concept: Training data, models, and predictions.
- Ethics: Bias in AI.

Action Item: Mini-Task: Play around with Quick, Draw! and discuss how it guesses the drawings.

Session 34 | Training Custom Models

Goal: Build a real Machine Learning model without code.

- Tool: Google Teachable Machine.
- Project: Gathering training data (images or sounds).
- Training the model and testing its accuracy.

Action Item: Homework: Train an Image model that can tell the difference between 3 of your favorite toys/objects.

Week 18: AI in the Web

Session 35 | Exporting AI to Code

Goal: Integrate the custom ML model into a website.

- Code: Exporting the Teachable Machine model as a Javascript snippet.
- Code: Importing the model into the Next.js app.
- Setting up the webcam in React.

Action Item: Mini-Task: Get the webcam working inside your Next.js project.

Session 36 | Smart Webcam App

Goal: Make the app react to AI predictions.

- Code: Running the video feed through the ML model.
- Code: Triggering UI changes based on predictions (e.g., if it sees Toy A, display 'Winner!').

Action Item: Homework: Complete the 'Smart Object Recognizer' web app.

Week 19: Python & Data Science Basics

Session 37 | Python Refresher & Google Colab

Goal: Transition to Python for data manipulation.

- Tool: Using Google Colab (Jupyter Notebooks).
- Review: Variables, loops, and lists in Python.
- Concept: Why Python is the king of AI.

Action Item: Mini-Task: Write a simple Python script in Colab that sorts a list of numbers.

Session 38 | Playing with Pandas

Goal: Learn to analyze data like a scientist.

- Library: Introduction to Pandas.
- Code: Loading a CSV file (e.g., Pokemon stats, game scores).
- Code: Filtering and finding the 'strongest' or 'highest' values.

Action Item: Homework: Load a dataset of superheroes and write code to find all heroes with 'Strength > 80'.

Week 20: Machine Learning Fundamentals

Session 39 | Predicting the Future

Goal: Understand basic predictive modeling.

- Concept: Linear Regression (drawing a line through dots).
- Library: Scikit-learn basics.
- Code: Training a simple model to predict game scores based on hours played.

Action Item: Mini-Task: Train a basic model in Colab and make a prediction.

Session 40 | AI Ethics & Safety

Goal: Understand the responsibilities of coding AI.

- Discussion: Deepfakes, hallucinations, and safety guardrails.
- Discussion: How to use AI responsibly as a young developer.
- Review of tools like ChatGPT/Gemini and how to write good prompts.

Action Item: Homework: Write a short paragraph or present on 'Why AI needs rules'.

Week 21: Grand Finale Project: Planning

Session 41 | Brainstorming the Masterpiece

Goal: Design an app that combines Flutter/Web with AI.

- Ideation: Choosing a problem to solve (e.g., AI Homework Helper, Smart Habit Tracker).
- Tech Stack choice: Will it be a mobile app or a web app?
- Defining the MVP (Minimum Viable Product).

Action Item: Mini-Task: Pitch the final project idea to the tutor.

Session 42 | UX Design & Architecture

Goal: Blueprint the entire application.

- Design: Drawing screens in Figma or on paper.
- Database Schema: Planning collections.
- API endpoints: Planning what the frontend will ask the backend.

Action Item: Homework: Finalize all designs and architectural plans.

Week 22: Grand Finale Project: The Backend

Session 43 | Setting up the Database & Server

Goal: Build the data foundation.

- Code: Initializing Node/Express server.
- Code: Connecting MongoDB and creating models.
- Testing the connection.

Action Item: Mini-Task: Ensure the backend runs and connects to the DB without errors.

Session 44 | Building Core APIs & AI Integrations

Goal: Create the routes and integrate AI models.

- Code: Writing CRUD routes.
- Code: Adding logic to call Gemini API or custom Python scripts.
- Testing with Postman.

Action Item: Homework: Complete all required backend routes so the API is fully functional.

Week 23: Grand Finale Project: The Frontend

Session 45 | Building the UI

Goal: Create the visual layer in Next.js or Flutter.

- Code: Translating Figma designs to code.
- Code: Using state management to handle data.
- Adding animations and styling.

Action Item: Mini-Task: Complete the 'Home' screen of the application.

Session 46 | Connecting the Dots

Goal: Hook up the frontend to the backend.

- Code: Making HTTP requests from the frontend to your custom API.
- Handling errors and loading states gracefully.
- End-to-end testing.

Action Item: Homework: Have a fully connected, functioning application.

Week 24: The Ultimate Presentation

Session 47 | Bug Hunting & Polish

Goal: Ensure the app is ready for the real world.

- QA Testing: Trying to break the app.
- Code cleanup: Removing console.logs and unused variables.
- Deployment: Pushing updates to Vercel/Render or preparing a physical device demo.

Action Item: Mini-Task: Fix at least 3 bugs found during testing.

Session 48 | Graduation Presentation

Goal: Celebrate achievements and present the final app.

- Live Demo: Walkthrough of the entire app.
- Code Walkthrough: Explaining the most complex parts (AI, Database).
- Review: Looking back at the 6-month journey and discussing next steps.

Action Item: Final Celebration! 🎉



Recommended Tools & Resources

- **Code Editor:** VS Code or Cursor (Cursor is excellent for AI-assisted pair programming).
- **Design:** Figma (Use community templates for wireframing) & Canva for App Icons.
- **Cloud & Hosting:** Vercel (Next.js frontend), Render (Node backend), MongoDB Atlas (Free Tier Database).
- **AI & ML Tools:** Google Teachable Machine, Quick Draw, Google Colab (for Python/Pandas).
- **APIs:** Google Gemini API or OpenAI API (Free tiers available for simple text generation).
- **Version Control:** GitHub (Continue building the portfolio started in previous classes).